SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 01 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO1: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class QUESTIONS: 5

Total Marks:50

Annexure A **CASE STUDY**

Code of Conduct:

I Dinesh V (192524309) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dinesh V

Annexure A

CASE STUDY

Q1. A compiler developer is designing a semantic analyzer to process arithmetic expressions and generate an internal representation using syntax trees. The analyzer must also define appropriate syntax-directed definitions (SDD) to compute and represent expressions correctly during parsing.

The developer is given the following expression:

a + b * c - d

The system must ensure:

Correct operator precedence (* > + > -)

Proper left associativity of operators Accurate construction of syntax tree
Definition of semantic rules for evaluation

Tasks:

(i) Construct the syntax tree for the given expression considering operator precedence and associativity.

(ii) Write the syntax-directed definition (SDD) for the grammar used to generate such expressions.

(iii) Show how attributes are evaluated for the given expression.

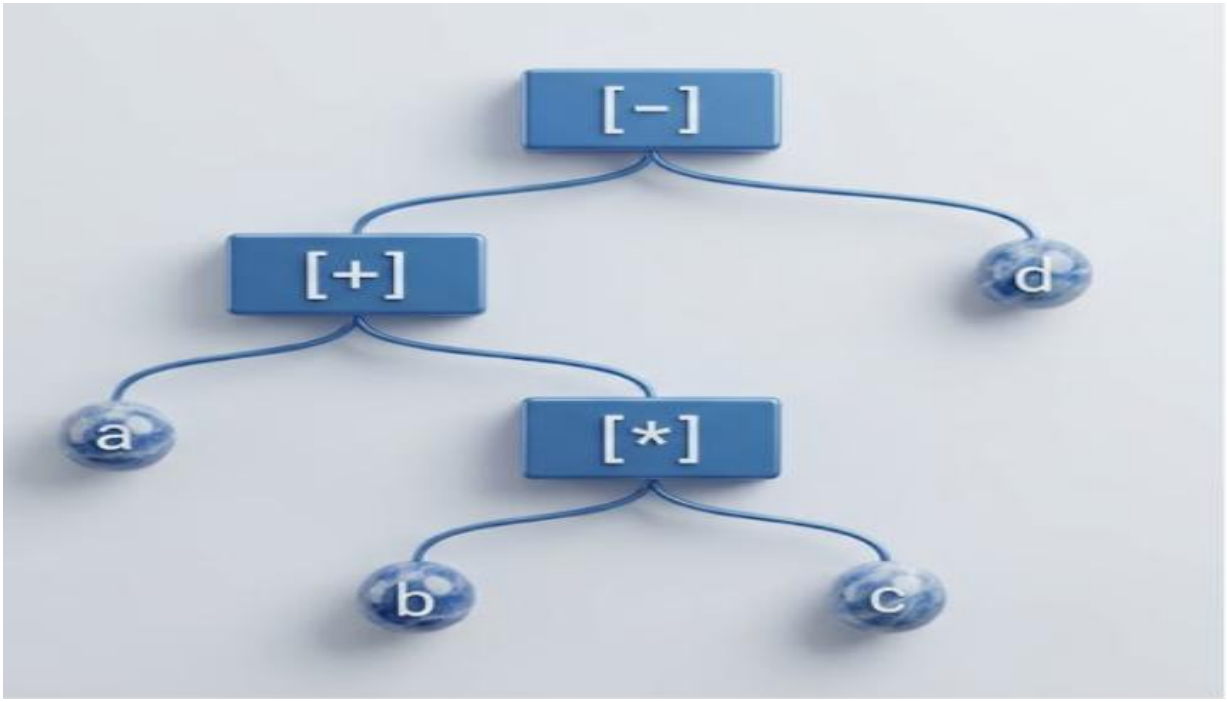
(iv) Explain how the syntax tree represents the correct order of operations. (10 Marks)

Ans:

(i) Syntax Tree for $a + b * c - d$

Considering operator precedence ($*> +> -$) and left associativity:

- $b * c$ is evaluated first.
- $a + (b * c)$ is evaluated second.
- $(a + b * c) - d$ is evaluated last.



(ii) Syntax-Directed Definition (SDD) for Syntax Tree Construction

Let $E \rightarrow E_1 + T$, $E \rightarrow E_1 - T$, $E \rightarrow T$, $T \rightarrow T_1 * F$, $T \rightarrow F$, $F \rightarrow$ id.

Production	**Semantic Rules**
$E \rightarrow E_1 + T$	$E.node =$ mknode('+', $E_1.node, T.node$)
$E \rightarrow E_1 - T$	$E.node =$ mknode('-', $E_1.node, T.node$)
$E \rightarrow T$	$E.node = T.node$
$T \rightarrow T_1 * F$	$T.node =$ mknode('*', $T_1.node, F.node$)
$T \rightarrow F$	$T.node = F.node$

Production	Semantic Rules
$F \rightarrow$ id	$F.node =$ mkleaf(id, id.entry)

(iii) Attribute Evaluation Steps

1. $F_{1.}node =$ mkleaf(id, a)
2. $T_{1.}node = F_{1.}node$
3. $E_{1.}node = T_{1.}node$ (Points to leaf a)
4. $F_{2.}node =$ mkleaf(id, b)
5. $T_{2.}node = F_{2.}node$
6. $F_{3.}node =$ mkleaf(id, c)
7. $T_{3.}node =$ mknode$('*', T_{2.}node, F_{3.}node)$ (Points to $b * c$)
8. $E_{2.}node =$ mknode$('+', E_{1.}node, T_{3.}node)$ (Points to $a + (b * c)$)
9. $F_{4.}node =$ mkleaf(id, d)
10. $T_{4.}node = F_{4.}node$
11. $E_{3.}node =$ mknode$('-', E_{2.}node, T_{4.}node)$ (Points to $\big(a + (b * c)\big) - d$)

(iv) Representation of Order of Operations

- **Precedence:** Lower-precedence operators $(+, -)$ are positioned higher in the tree than higher-precedence operators $(*)$. Since evaluation happens bottom-up (post-order traversal), higher-precedence nodes $(*)$ are evaluated first.
- **Associativity:** Left-associative operators are constructed as left-heavy subtrees, ensuring operations are grouped from left to right.

Q2.A compiler developer is implementing a semantic analyzer for arithmetic expressions using S-attributed syntax-directed definitions (SDD). The system is designed to evaluate expressions and simultaneously generate intermediate code using a stack-based parser (bottom-up evaluation).

The following grammar is used to define arithmetic expressions:

S → E n

E → E + T | E - T | T

T → T * F | T / F | F

F → (E) | digit

The input string to be processed is:

8 + 2 * 4 n

The compiler must:

Evaluate the expression using bottom-up parsing

Apply S-attributed definitions

Generate corresponding intermediate code (translator output)

Tasks:

(i) Construct the S-attributed syntax-directed definition (SDD) for the given grammar, specifying semantic rules for each production.

(ii) Develop a code fragment (translator) to evaluate expressions using the defined SDD.

(iii) Use a parser stack to demonstrate the step-by-step evaluation of the input string:

8 + 2 * 4 n

Show stack contents

Show intermediate computations

(iv) Construct the syntax tree for the given expression and annotate it with computed values.

(v) Explain how bottom-up evaluation using S-attributed definitions ensures correct operator precedence and associativity. (10 Marks)
Ans:

(i) S-Attributed SDD

Production	Semantic Rules	Translator Code Action
$S \rightarrow En$	$S.val = E.val$	print(E.val)
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$	emit('+', E1.val, T.val)
$E \rightarrow E_1 - T$	$E.val = E_1.val - T.val$	emit('-', E1.val, T.val)
$E \rightarrow T$	$E.val = T.val$	—
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$	emit('*', T1.val, F.val)
$T \rightarrow T_1/F$	$T.val = T_1.val/F.val$	emit('/', T1.val, F.val)
$T \rightarrow F$	$T.val = F.val$	—
$F \rightarrow (E)$	$F.val = E.val$	—
$F \rightarrow$ digit	$F.val =$ digit.lexval	—

(ii) Code Fragment (Translator)

C

```
void evaluate_and_translate(int production_id) {
    switch(production_id) {
        case 1: // E -> E1 + T
            stack[top - 2].val = stack[top - 2].val + stack[top].val;
            printf("t1 = %d + %d\n", stack[top - 2].val, stack[top].val);
```

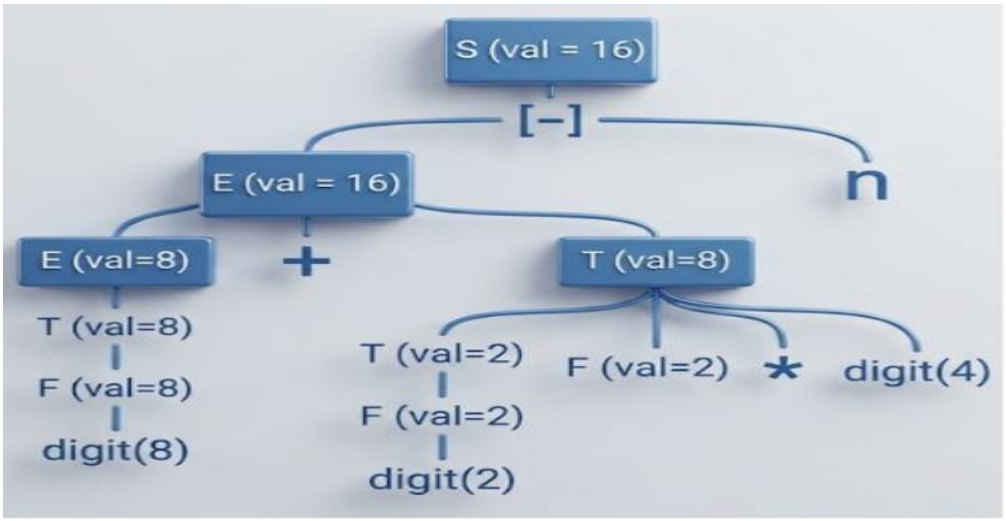
```
            top -= 2; break;
        case 2: // T -> T1 * F
            stack[top - 2].val = stack[top - 2].val * stack[top].val;
            printf("t2 = %d * %d\n", stack[top - 2].val, stack[top].val);
            top -= 2; break;
        // Additional cases follow same pattern
    }
}
```

(iii) Parser Stack Tracing for 8 + 2 * 4 n

Stack	**Input**	**Action**	**Attribute Computation / Output**
$	8 + 2 * 4 n	Shift 8	—
$ 8	+ 2 * 4 n	Reduce $F \rightarrow$ digit	$F.val = 8$
$ F(8)	+ 2 * 4 n	Reduce $T \rightarrow F$	$T.val = 8$
$ T(8)	+ 2 * 4 n	Reduce $E \rightarrow T$	$E.val = 8$
$ E(8)	+ 2 * 4 n	Shift +	—
$ E(8) +	2 * 4 n	Shift 2	—
$ E(8) + 2	* 4 n	Reduce $F \rightarrow$ digit	$F.val = 2$

Stack	Input	Action	Attribute Computation / Output
$ E(8) + F(2)	* 4 n	Reduce $T \rightarrow F$	$T.val = 2$
$ E(8) + T(2)	* 4 n	Shift *	—
$ E(8) + T(2) *	4 n	Shift 4	—
$ E(8) + T(2) * 4	n	Reduce $F \rightarrow$ digit	$F.val = 4$
$ E(8) + T(2) * F(4)	n	Reduce $T \rightarrow T * F$	$T.val = 2 \times 4 = 8$; Output: t1 = 2 * 4
$ E(8) + T(8)	n	Reduce $E \rightarrow E + T$	$E.val = 8 + 8 = 16$; Output: t2 = 8 + 8
$ E(16)	n	Shift n	—
$ E(16) n	$	Reduce $S \rightarrow En$	$S.val = 16$; Output: Result = 16

(iv) Annotated Syntax Tree



(v) Ensured Precedence and Associativity

- **Precedence:** In bottom-up parsing, $2 * 4$ is reduced before $E + T$ is reduced because $T \rightarrow T * F$ has higher precedence in grammar rules.

- **Associativity:** Left-recursive grammar rules ($E \rightarrow E + T$) force reductions to happen left-to-right, ensuring correct evaluation order.

Q3. A compiler developer is designing a semantic analyzer using syntax-directed definitions (SDD). The analyzer must determine whether given attribute rules follow valid evaluation strategies such as S-attributed or L-attributed definitions, and whether they can be evaluated without conflicts.

Consider the production:

A → B C D

Each non-terminal (A, B, C, D) has two attributes:

s → synthesized attribute i →
inherited attribute

The developer is given the following sets of semantic rules:

Set (i):

A.s = B.i + C.i Set
(ii):
A.s = B.i + C.s

D.i = A.i + B.s Set
(iii):
A.s = B.s + D.s Set
(iv):
A.s = D.i

B.i = A.s + D.s

C.i = B.s

D.i = B.i + C.i

Tasks:

(i) For each set of rules, determine whether they are consistent with an S-attributed definition.

(ii) Determine whether the rules satisfy the conditions of an L-attributed definition.

(iii) Analyze whether the rules allow a valid evaluation order (i.e., no circular dependency).

(iv) For each set, justify your answer by explaining the attribute dependency relationships.

(v) Identify any circular dependencies and explain why they prevent evaluation.(10 Marks)

Ans:

(i) & (ii) Analysis Table

Set	Rules	S-Attributed?	L-Attributed?
Set (i)	$A.s = B.i + C.i$	**No** (Uses inherited attributes $B.i, C.i$)	**No** ($B.i$ and $C.i$ are used without prior definition)
Set (ii)	$A.s = B.i + C.s$, $D.i = A.i + B.s$	**No** (Uses inherited attributes $B.i, D.i, A.i$)	**Yes** ($D.i$ depends on parent $A.i$ and left-sibling $B.s$)
Set (iii)	$A.s = B.s + D.s$	**Yes** (Only synthesizes head attribute from children)	**Yes** (Every S-attributed SDD is also L-attributed)
Set (iv)	$A.s = D.i$, $B.i = A.s + D.s$, $C.i = B.s$, $D.i = B.i + C.i$	**No**	**No** (Contains circular dependencies)

(iii), (iv) & (v) Circular Dependency Analysis

- **Set (i):** Invalid if $B.i$ and $C.i$ are undefined.
- **Set (ii):** Valid order exists (Top-down for inherited, bottom-up for synthesized).
- **Set (iii):** Perfectly valid; standard synthesized evaluation order.
- **Set (iv) Circular Dependency:**

$$A.s \longrightarrow D.i \longrightarrow B.i \longrightarrow D.i$$

- $A.s$ depends on $D.i$.
- $B.i$ depends on $A.s$.
- $D.i$ depends on $B.i$.
- **Conclusion:** This forms a direct directed cycle ($A.s \rightarrow D.i \rightarrow B.i \rightarrow A.s$). Circular dependencies make evaluation impossible because no node in the cycle can compute its value first.

Q4. A compiler developer is designing a semantic analyzer for a programming language that supports variable declarations. During semantic analysis, the compiler must propagate type information from a type specifier to a list of identifiers and update the symbol table accordingly. The compiler uses the following grammar:

D → T L

T → int | real

L → L , id | id

The input declaration is:

real p, q, r

The semantic analyzer must perform bottom-up evaluation of inherited attributes while parsing the declaration and ensure that the type information is correctly assigned to all identifiers.
Tasks

(i). Construct a suitable Syntax-Directed Definition (SDD) for the given grammar to propagate type information from T to all identifiers in L.
(ii). Identify the inherited and synthesized attributes used in the grammar and explain their roles.
(iii). Demonstrate the bottom-up evaluation process for the input string:

real p, q, r showing how attribute values are computed during reductions.
(iv). Construct the annotated parse tree and clearly indicate the attribute values associated with each node.
(v). Illustrate the stack implementation of bottom-up parsing, showing:

Stack contents

Remaining input

Parser action (Shift/Reduce)

Attribute evaluation performed at each reduction (10 Marks)

(i) Syntax-Directed Definition (SDD)

Production	Semantic Rules
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow$ real	$T.type =$ real
$T \rightarrow$ int	$T.type =$ int
$L \rightarrow L_1$,id	$L_1.inh = L.inh$, addtype(id.entry, $L.inh$)
$L \rightarrow$ id	addtype(id.entry, $L.inh$)

(ii) Attribute Classification

- $T.type$ **(Synthesized):** Passes type information up from terminal keywords (real/int).
- $L.inh$ **(Inherited):** Passes type information down and across to identifier list elements.

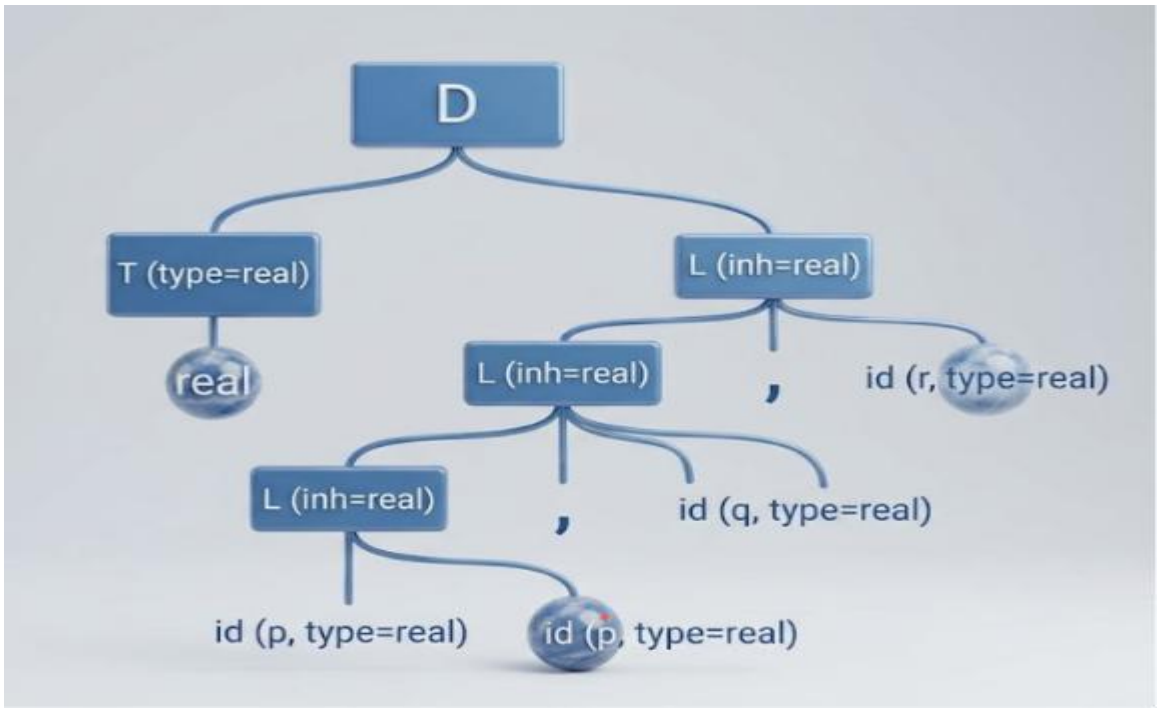
(iii) & (v) Stack-Based Bottom-Up Evaluation Trace

To evaluate inherited attributes in bottom-up parsing, marker non-terminals or a copy on the stack is used:

Stack	Input	Action	Attribute Actions
$	real p, q, r	Shift real	—
$ real	p, q, r	Reduce $T \rightarrow$ real	$T.type =$ real
$ T(real)	p, q, r	Shift p	—
$ T(real) p	, q, r	Reduce $L \rightarrow$ id	Inherit type from T; addtype(p,real)
$ T(real) L	, q, r	Shift ,	—

Stack	Input	Action	Attribute Actions
$ T(real) L ,	q, r	Shift q	—
$ T(real) L , q	, r	Reduce $L \rightarrow L$,id	addtype(q,real)
$ T(real) L	, r	Shift ,	—
$ T(real) L ,	r	Shift r	—
$ T(real) L , r	$	Reduce $L \rightarrow L$,id	addtype(r,real)
$ T(real) L	$	Reduce $D \rightarrow TL$	Declaration complete

(iv) Annotated Parse Tree



Q5. A compiler developer is designing a semantic analyzer that performs top-down translation of arithmetic expressions using Syntax-Directed Definitions (SDD). The analyzer must evaluate arithmetic expressions and construct an annotated parse tree during parsing.
The following grammar is used:

S → E n

E → E + T | T

T → T * F | F

F → (E) | digit

The input expression is:

3 * 4 + 6 n

The developer must ensure:

Proper top-down evaluation using L-attributed definitions

Construction of an annotated parse tree

Correct computation of expression values

Proper handling of operator precedence and associativity

Tasks

(i). Construct a suitable Syntax-Directed Definition (SDD) for the given grammar using L-attributed definitions.
(ii). Show how inherited and synthesized attributes are propagated during top-down parsing.
(iii). Perform the top-down translation for the input string:

3 * 4 + 6 n and compute the final expression value.

(iv). Construct the annotated parse tree, clearly indicating attribute values at each node. (10 Marks)

(i) L-Attributed SDD

Production	**Semantic Rules**
$S \rightarrow En$	$S.val = E.val$
$E \rightarrow TE'$	$E'.inh = T.val, E.val = E'.syn$
$E' \rightarrow +TE'_1$	$E'_1.inh = E'.inh + T.val, E'.syn = E'_1.syn$
$E' \rightarrow \epsilon$	$E'.syn = E'.inh$

Production	Semantic Rules
$T \rightarrow FT'$	$T'.inh = F.val, T.val = T'.syn$
$T' \rightarrow * FT'_1$	$T'_{1.}inh = T'.inh \times F.val, T'.syn = T'_{1.}syn$
$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
$F \rightarrow$ digit	$F.val =$ digit.lexval

(ii) Attribute Propagation

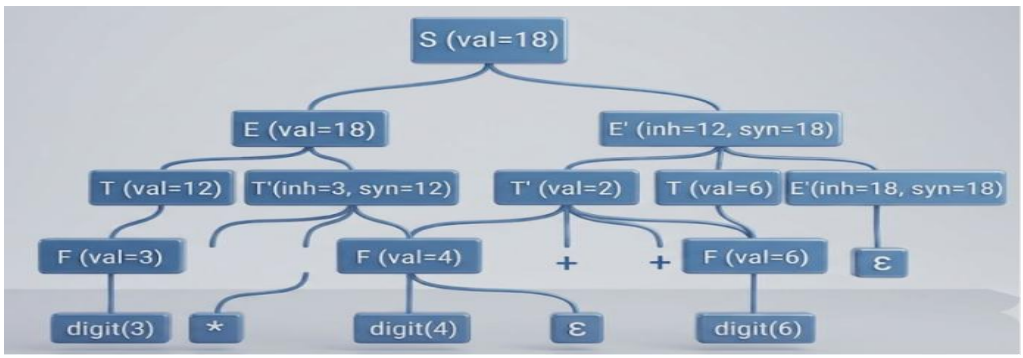
1. **Inherited attributes (inh):** Computed top-down and left-to-right to pass intermediate calculations down the tree.
2. **Synthesized attributes (syn/val):** Computed bottom-up to pass final values back up to the root.

(iii) Evaluation Process for $3 * 4 + 6n$

1. $F_{1.}val = 3$
2. $T'_{1.}inh = 3$
3. $F_{2.}val = 4$
4. $T'_{2.}inh = 3 \times 4 = 12$
5. $T'_{2.}syn = 12 \Longrightarrow T'_{1.}syn = 12 \Longrightarrow T_{1.}val = 12$
6. $E'_{1.}inh = 12$
7. $F_{3.}val = 6 \Longrightarrow T_{2.}val = 6$
8. $E'_{2.}inh = 12 + 6 = 18$
9. $E'_{2.}syn = 18 \Longrightarrow E'_{1.}syn = 18 \Longrightarrow E.val = 18$

Final Output Value = 18

(iv) Annotated Parse Tree



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand